CAREER OS: AI-POWERED JOB PREPARATION AND CAREER READINESS PLATFORM

PROJECT REPORT

Submitted by

UOIKKATTAN S (714025243236)
THARAN R (714025243228)
THAMIZHARASAN M (714025243224)
VAISHNAVI S (714025243239)

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SRI SHAKTHI INSTITUTE OF ENGINEERING AND TECHNOLOGY

COIMBATORE – 641 062

An Autonomous Institute,

Accredited by NAAC with "A" Grade

ANNA UNIVERSITY: CHENNAI 600 025

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "**CAREER OS: AI-POWERED JOB PREPARATION AND CAREER READINESS PLATFORM**" is the bonafide work of "**UOIKKATTAN S (714025243236), THARAN R (714025243228), THAMIZHARASAN M (714025243224), VAISHNAVI S (714025243239)**" who carried out the project work under my supervision.

SIGNATURE

Dr. KAMALAVENI V
M.E., Ph.D.
PROJECT GUIDE
Assistant Professor,
Department of Artificial Intelligence and Data Science,
Sri Shakthi Institute of Engineering and Technology, Coimbatore – 641 062.

SIGNATURE

Dr. N. M. SARAVANA KUMAR
M.E., Ph.D.
HEAD OF THE DEPARTMENT
Department of Artificial Intelligence and Data Science,
Sri Shakthi Institute of Engineering and Technology, Coimbatore – 641 062.

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

First and foremost, we thank God Almighty for giving us the strength, wisdom and determination to complete this project successfully. Without His blessings and guidance, this achievement would not have been possible.

We express our deepest gratitude to our Chairman, **Dr. S. Thangavelu**, for his continuous encouragement, valuable support and commitment towards providing quality education and a supportive academic environment throughout our course of study.

We are thankful to our Secretary, **Dr. T. Dheepan**, and our Joint Secretary, **Mr. T. Sheelan**, for their constant support and for providing the necessary facilities to carry out this project work successfully.

We are highly indebted to our Principal, **Dr. N. M. Saravana Kumar, M.E., Ph.D.**, for his valuable support, encouragement and for creating a favourable learning environment during the tenure of this project.

We express our sincere thanks to our Head of the Department, **Dr. N. M. Saravana Kumar**, for his encouragement, guidance and valuable suggestions, which helped us in the systematic planning and execution of our project work.

It is our great pleasure to thank our Project Guide and Project Coordinator, **Dr. Kamalaveni V.**, Assistant Professor, Department of Artificial Intelligence and Data Science, for her valuable technical suggestions, continuous guidance, constructive feedback and encouragement throughout the development of our project, "CAREER OS: AI-POWERED JOB PREPARATION AND CAREER READINESS PLATFORM."

We also extend our sincere thanks to all the teaching and non-teaching staff members of the Department of Artificial Intelligence and Data Science, and to our friends and classmates, for their cooperation, motivation and support during the different stages of this project.

Finally, we express our heartfelt gratitude to everyone who directly or indirectly contributed to the successful completion of this project.

UOIKKATTAN S (714025243236)
THARAN R (714025243228)
THAMIZHARASAN M (714025243224)
VAISHNAVI S (714025243239)

ABSTRACT

Students preparing for jobs usually depend on separate tools for resume writing, aptitude practice, interview rehearsal and job search, and none of these tools tells a student how ready they are or what to do next. Career OS is a web-based platform that brings these activities together and gives feedback computed from the student's own saved data. The system uses a Next.js and TypeScript frontend, a FastAPI backend and a relational database accessed through SQLAlchemy.

A student can upload a resume that is analysed by documented rule-based formulas, practise from a reviewed bank of 235 questions, train for any job role by choosing or typing a title, receive immediate marks and explanations, and read a complete scored review of every session. A role engine maps any job title to practice topics, prefers questions the student has not seen, adapts difficulty to earlier results and, when a language-model key is configured, generates validated questions to fill gaps. An AI Mentor answers free-form questions through a hosted language model using only verified facts from the student's own records. Recruiters, placement officers and administrators have separate portals.

The backend exposes 124 routes and has 483 automated tests, of which 481 passed in the final run; the frontend has 54 passing tests. No model was trained, and persistence on the remote database was not verified.

Keywords: job preparation, resume analysis, adaptive assessment, role-based training, large language model, FastAPI, Next.js.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ACKNOWLEDGEMENT	**iii**
	ABSTRACT	**iv**
	LIST OF TABLES	**vii**
	LIST OF FIGURES	**viii**
	LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE	**ix**
1.	**INTRODUCTION**	**1**
	1.1 BACKGROUND OF THE PROJECT	1
	1.2 OVERVIEW OF CAREER OS	1
	1.3 PROBLEM STATEMENT	2
	1.4 OBJECTIVES OF THE PROJECT	2
	1.5 SCOPE AND MOTIVATION	2
	1.6 EXISTING SYSTEM AND PROPOSED SYSTEM	3
2.	**LITERATURE SURVEY**	**4**
	2.1 JOB PREPARATION AND INTERVIEW PLATFORMS	4
	2.2 RESUME SCREENING SYSTEMS	4
	2.3 ONLINE ASSESSMENT SYSTEMS	4
	2.4 AI-BASED CAREER ASSISTANCE	4
	2.5 COMPARATIVE ANALYSIS AND RESEARCH GAP	5
3.	**SYSTEM REQUIREMENTS**	**6**
	3.1 OVERVIEW OF SYSTEM REQUIREMENTS	6
	3.2 SOFTWARE AND HARDWARE REQUIREMENTS	6
	3.3 FUNCTIONAL REQUIREMENTS	6
	3.4 NON-FUNCTIONAL REQUIREMENTS	7
	3.5 FEASIBILITY, CONSTRAINTS AND ASSUMPTIONS	7
4.	**PROPOSED METHODOLOGY**	**8**
	4.1 METHODOLOGY OVERVIEW	8
	4.2 OVERALL SYSTEM ARCHITECTURE	8
	4.3 AUTHENTICATION AND AI PROCESSING FLOW	10
	4.4 MODULE DESCRIPTION	12
	4.5 DATA FLOW, USE CASE AND ACTIVITY DIAGRAMS	12
	4.6 ENTITY RELATIONSHIP DIAGRAM	14
	4.7 ALGORITHMS AND PSEUDOCODE	15
5.	**IMPLEMENTATION**	**17**
	5.1 IMPLEMENTATION OVERVIEW	17
	5.2 DATABASE IMPLEMENTATION	17
	5.3 BACKEND API IMPLEMENTATION	17
	5.4 RESUME ANALYSIS PIPELINE	18

	5.5	USER INTERFACE AND SCREENS	18
	5.6	TESTING STRATEGY	25
6.	**RESULTS AND DISCUSSION**		**27**
	6.1	STATUS OF RESULTS	27
	6.2	TEST EXECUTION RESULTS	27
	6.3	DISCUSSION OF THE FINDINGS	28
	6.4	ADVANTAGES AND LIMITATIONS	28
7.	**CONCLUSION AND FUTURE ENHANCEMENTS**		**29**
	7.1	CONCLUSION	29
	7.2	FUTURE ENHANCEMENTS	29
8.	**REFERENCES**		**30**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	Comparison of Existing Approaches and the Proposed System	3
2.1	Literature Survey Comparison	5
2.2	Research Gap Analysis	5
3.1	Software and Hardware Requirements	6
3.2	Functional Requirements	6
3.3	Non-Functional Requirements and How They Are Met	7
4.1	Summary of the Career OS Modules	12
4.2	Classification of Intelligent Features	15
4.3	Marking Rules of Role-Based Training	16
5.1	Technologies and Their Roles	17
5.2	Key API Endpoints	18
5.3	Test Cases and Expected Outcomes	26
6.1	Summary of Verification Results (Final Run, 31 October 2026)	27
6.2	Known Limitations	28
7.1	Current Limitations and Proposed Future Enhancements	29

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	Main Modules of the Career OS Student Workspace	1
4.1	Overall System Architecture of Career OS	9
4.2	Layered Architecture of the Backend	10
4.3	Authentication and Authorization Flow	11
4.4	Processing Flow of an AI Mentor Message	11
4.5	Data Flow Diagram, Level 1	13
4.6	Use Case Diagram of Career OS	13
4.7	Activity Diagram of a Role-Based Training Session	14
4.8	Entity Relationship Diagram of the Core Tables	15
5.1	Resume Analysis Pipeline	18
5.2	Login Page (Left) and Registration Page (Right)	19
5.3	Student Dashboard	19
5.4	Resume Center with the Rule-Based Analysis	20
5.5	Interview Preparation Home with Progress by Round	20
5.6	Role Selection with a Custom Job Title	21
5.7	Immediate Feedback After Submitting an Answer	21
5.8	Assessment Review: Score, Counts and Topic-Wise Performance	22
5.9	Question-by-Question Review with Model Answer and Missing Concepts	22
5.10	AI Mentor Conversation with Suggested Next Steps	23
5.11	Recommended IT Jobs from a Live Remote-Jobs Source	23
5.12	Communication Coach Scenario Selection	24
5.13	Career Discovery Questionnaire	24
5.14	Learning Roadmap in the Training Academy	25
5.15	Recruiter Portal (Left) and Placement Officer Portal (Right)	25
5.16	Administrator Console	25
6.1	Number of Backend Tests in Each Layer	27

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

ABBREVIATION	DESCRIPTION
AI	Artificial Intelligence
API	Application Programming Interface
ATS	Applicant Tracking System
CORS	Cross-Origin Resource Sharing
DFD	Data Flow Diagram
ER	Entity Relationship
HTTP(S)	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MCQ	Multiple-Choice Question
OCR	Optical Character Recognition
ORM	Object-Relational Mapping
PDF	Portable Document Format
RBAC	Role-Based Access Control
REST	Representational State Transfer
SQL	Structured Query Language
UI / UX	User Interface / User Experience
UUID	Universally Unique Identifier
WS	WebSocket

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND OF THE PROJECT

Preparing for a first job is a multi-step task. A student has to write a resume that a screening system can read, practise aptitude and technical questions, learn to speak about projects, rehearse interviews and find openings that match their skills. Most students handle each step with a different website, and the results are never connected: a poor quiz score does not change what the student practises next, and a resume score does not influence the interview preparation. Practice material is also generic; a student applying for a backend developer position and one applying for a cloud engineer position are usually shown the same question lists. Large studies of hiring practices report that automated screening can filter out qualified applicants because of rigid rules [1], so students also need to understand how machines read their documents.

1.2 OVERVIEW OF CAREER OS

Career OS is a web-based job-preparation and career-readiness platform developed to close this gap. It keeps the student's profile, resume analysis, practice results, interview reports and goals in one database and uses them to produce feedback and recommendations for four kinds of users: students, recruiters, placement officers and administrators (Fig. 1.1).

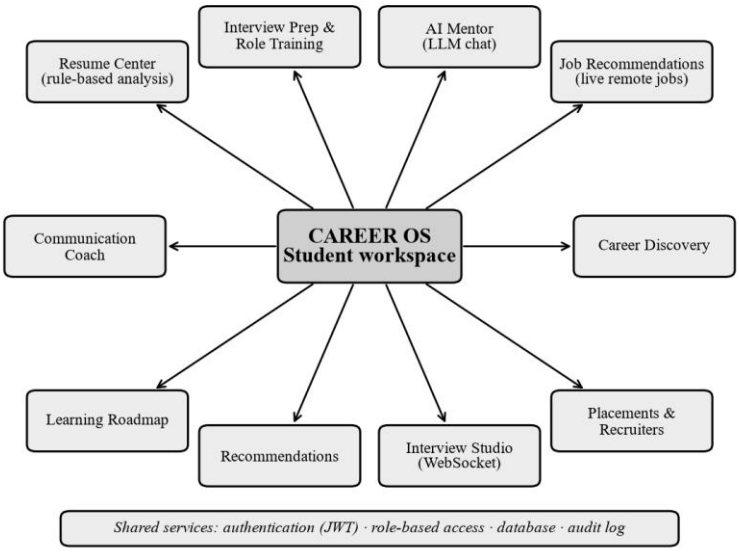


Fig. 1.1: Main Modules of the Career OS Student Workspace

The system is deliberately honest about its intelligence. Resume analysis, interview and communication feedback, recommendations, the roadmap and placement ranking are rule-based algorithms with documented formulas. A large language model is used in the AI Mentor and, optionally, to generate and grade role-specific questions when a provider key is configured. No machine-learning model is trained in this project.

1.3 PROBLEM STATEMENT

Students lack a single place that (i) measures how complete and readable their resume is, (ii) lets them practise questions that match the job role they want, however unusual the title, (iii) gives immediate and explained feedback on every answer, (iv) remembers what they have already practised so that new sessions contain fresh questions, and (v) answers their free-form questions without inventing facts about them. The problem addressed by Career OS is the design and implementation of such a platform with honest, traceable scoring.

1.4 OBJECTIVES OF THE PROJECT

1. To provide secure registration and login with role-based access for four user groups.
2. To analyse an uploaded resume and report completeness, readability and wording with transparent formulas.
3. To let a student practise for any job role, prefer unseen questions, adapt difficulty and explain the choice.
4. To evaluate every answer immediately and compute final scores on the server from saved records.
5. To provide an AI Mentor that uses only verified facts from the student’s account, and portals for recruiters, officers and administrators.
6. To verify the implementation with automated tests and report its limitations honestly.

1.5 SCOPE AND MOTIVATION

The scope covers the web application, the backend services, the database design and the automated tests. It excludes a mobile application, payments, e-mail delivery, social sign-in, speech or video analysis and execution of student-written code. A college placement cell can use the platform to prepare students before campus drives and to rank registered students; a training institute can use the role engine for roles that standard question sets do not cover; an individual job seeker can use the resume analysis and the mentor to find missing skills. Scores are practice feedback, not predictions of hiring decisions.

1.6 EXISTING SYSTEM AND PROPOSED SYSTEM

The usual approach is a collection of independent tools: a word-processor template for the resume, free online quizzes, video tutorials for interviews, a job portal and a general chatbot. Each has its own login, its own data and its own idea of progress, so practice is generic, feedback is unexplained and chatbots may state invented details about the user. Career OS is proposed as an integrated platform with a web frontend, an application server and a relational database. Table 1.1 compares the two.

Table 1.1: Comparison of Existing Approaches and the Proposed System

Aspect	Existing approaches (separate tools)	Proposed Career OS
Readiness view	Tools are disconnected; no overall view	One database; recommendations read resume, practice and goals together
Role relevance	Not role-specific; questions repeat	Role engine maps any job title to topics and prefers unseen questions
Feedback	Delayed, vague or unexplained	Immediate result, correct answer, explanation and suggestion, labelled by method
Trust in the assistant	General chatbots may invent facts about the user	Mentor receives only verified records; refuses without a configured provider
Auditability	Little or no evidence trail	Scores computed on the server from saved records

CHAPTER 2

LITERATURE SURVEY

2.1 JOB PREPARATION AND INTERVIEW PLATFORMS

Preparation material is delivered through job portals that list vacancies, assessment sites that offer timed quizzes and coding problems, interview-practice services and learning platforms. Job portals concentrate on search and applications, and assessment sites on a limited set of skills. Question lists are usually static, so a student who practises regularly sees the same questions again, and none of these channels combines resume feedback, role-specific practice and a conversational assistant under one account.

2.2 RESUME SCREENING SYSTEMS

Employers use applicant tracking systems to store and filter applications. Fuller et al. [1] studied hiring practices in several countries and concluded that automated systems relying on rigid exclusion rules can screen out candidates who could do the job. Keyword matching, the basic technique behind many such checks, is related to term weighting in information retrieval [2] and to set-overlap measures such as the Jaccard index [3]. Career OS uses transparent keyword and structure measurements and states that its scores are not those of any employer's system.

2.3 ONLINE ASSESSMENT SYSTEMS

Roediger and Karpicke [4] showed that taking a test improves long-term retention compared with repeated study, and Black and Wiliam [5] reviewed evidence that formative assessment raises achievement. These findings motivate the design of Career OS, where every answer is graded immediately and explained and weak topics are revisited. Multiple-choice questions can be graded exactly, whereas free-text answers need a rubric; Career OS uses an expected-concept rubric and, when available, a language model to judge semantic equivalence.

2.4 AI-BASED CAREER ASSISTANCE

Large language models are based on the Transformer architecture [6]. Brown et al. [7] showed that a large model can perform new tasks from examples placed in the prompt, Wei et al. [8] described chain-of-thought prompting, and Lewis et al. [9] introduced retrieval-augmented generation, in which a model is grounded in retrieved text. These ideas explain

why Career OS supplies verified facts from the student's own records to the mentor instead of relying on the model's memory. Because such models can produce fluent but wrong statements, the platform restricts the context, caps the output and warns the student.

2.5 COMPARATIVE ANALYSIS AND RESEARCH GAP

Table 2.1: Literature Survey Comparison

Ref.	Work (author, year)	Main contribution	Relevance to this project
[1]	Fuller et al. (2021)	Rigid screening rules exclude capable candidates	Machine-readable resume feedback
[2], [3]	Salton and Buckley (1988); Jaccard (1912)	Term weighting; set-overlap measure	Keyword and skill-overlap matching
[4]	Roediger and Karpicke (2006)	Testing improves long-term retention	Immediate graded practice
[5]	Black and Wiliam (1998)	Formative assessment raises achievement	Explained feedback, weak-topic revisits
[6]	Vaswani et al. (2017)	Transformer architecture	Basis of the hosted language models
[7], [8]	Brown et al. (2020); Wei et al. (2022)	Few-shot and chain-of-thought prompting	Behaviour adapted by prompt, not training
[9]	Lewis et al. (2020)	Retrieval-augmented generation	Mentor grounded in verified records

The strands above are seldom combined in one preparation platform that is both personalised and trustworthy. Table 2.2 lists the gaps and the response of this project.

Table 2.2: Research Gap Analysis

No.	Gap identified in existing work	Response in the proposed system
1	Tools are disconnected and keep separate data	One database feeding resume, practice, mentor and recommendations
2	Question sets are static or role-agnostic	Role engine for any job title; unseen-question preference; validated AI generation
3	Chatbots may invent facts about the user	Mentor receives only verified facts and refuses to answer without a provider
4	Scores are unexplained	Documented formulas and method labels; server-side totals from saved records
5	Few tools report their own limits	Honest classification of features and a verified test record

Contribution of this work. The contribution is not a new language model but a verification-first, record-based design that integrates resume analysis, adaptive role-based practice, an evidence-grounded mentor and role portals in one auditable platform.

CHAPTER 3

SYSTEM REQUIREMENTS

3.1 OVERVIEW OF SYSTEM REQUIREMENTS

Students are the primary users: they maintain a profile, upload a resume, practise and consult the mentor. Recruiters publish jobs and move applicants through stages; placement officers manage campus drives, eligibility and ranking for their own college; administrators manage users, roles, announcements and the audit trail. Two external services are used: a language-model provider (Groq, xAI or Anthropic) and the public Himalayas remote-jobs interface.

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

Table 3.1: Software and Hardware Requirements

Category	Requirement
Operating system	Windows 11 (development); Linux container (deployment)
Languages and runtimes	Python 3.12 in the Dockerfile (3.14 on the development machine); Node.js 24; TypeScript 5
Backend frameworks	FastAPI with Uvicorn; Pydantic 2; SQLAlchemy 2 (async)
Frontend frameworks	Next.js 15.5 with React 19.1; Tailwind CSS 4; React Query 5; Zustand 5
Database	PostgreSQL (Supabase) for production; SQLite for local development and tests
Language model	Optional account with Groq (model openai/gpt-oss-120b), xAI or Anthropic
Tools	Git, npm, pip, pytest, Vitest, ESLint, ruff; a current web browser
Hardware (development)	Dual-core 2 GHz processor, 8 GB RAM, 5 GB free storage, internet access
Hardware (deployment, estimated)	2 virtual CPUs, 4 GB RAM, 20 GB storage, public HTTPS endpoint

3.3 FUNCTIONAL REQUIREMENTS

Table 3.2: Functional Requirements

ID	Requirement	Status
FR1	Register, log in, refresh a session; maintain a profile and target role	Implemented, tested
FR2	Upload a PDF or DOCX resume, analyse it and download the stored file	Implemented, rule-based
FR3	Practise aptitude, technical, coding, communication, HR and managerial rounds	Implemented, tested
FR4	Practise for any job role with fresh questions and a scored review	Implemented, tested
FR5	Run live interview and communication sessions over WebSocket	Implemented, rule-based
FR6	Chat with an AI Mentor and keep the conversation history	Implemented; needs an LLM key

ID	Requirement	Status
FR7	Suggest next steps and a learning roadmap; list recommended remote jobs	Implemented; jobs need internet
FR8	Recruiters: companies, jobs, applications. Officers: drives, eligibility, ranking	Implemented
FR9	Administration, community, dashboards and notifications	Implemented (partly basic)

3.4 NON-FUNCTIONAL REQUIREMENTS

Table 3.3: Non-Functional Requirements and How They Are Met

Quality	How it is met
Security and privacy	bcrypt hashes, JWT, role and ownership checks, login throttling, upload validation
Reliability	Typed provider errors; nothing is saved when an external service fails
Performance	Asynchronous server, React Query caching, bounded conversation history
Maintainability	Router, service, repository and model layers; 124 documented routes
Honesty and portability	Scores labelled by method; runs on SQLite locally and on PostgreSQL

3.5 FEASIBILITY, CONSTRAINTS AND ASSUMPTIONS

Technical feasibility. Every component uses mature open-source technology; the backend and frontend were built, tested and run locally, and a real language-model request through the configured provider was verified. **Operational feasibility.** Users need only a web browser, and the four user groups have separate portals. **Economic feasibility.** All frameworks are free software; the recurring costs are hosting and optional LLM usage, which is kept low by using stored questions first and by per-user limits. Secrets are supplied only through environment variables; resume uploads are PDF or DOCX files of at most 5 MB; reviewed questions exist for 22 catalogued roles; the system supports English text input only.

CHAPTER 4

PROPOSED METHODOLOGY

4.1 METHODOLOGY OVERVIEW

The proposed methodology is a layered, record-based design. A student's actions are stored as records, and every score, recommendation and mentor answer is computed from those saved records. The design follows five principles: (1) **records before replies**, every figure is traceable to saved data; (2) **rules before models**, resume, interview, recommendation and ranking logic use documented formulas; (3) **validate model output**, anything a language model returns is bounded, schema-checked and labelled; (4) **ownership everywhere**, each personal resource is filtered by the signed-in user; and (5) **fail safely**, an external failure produces a clear message and saves nothing.

4.2 OVERALL SYSTEM ARCHITECTURE

Career OS follows a three-tier client-server architecture (Fig. 4.1). The client tier is a Next.js application in the browser; all network access goes through one typed client, lib/api.ts, which attaches the access token, refreshes it once on a 401 response and converts failures into readable errors [10–13]. The application tier is a FastAPI server that validates requests, enforces authentication and authorization and applies business rules. The data tier is a relational database accessed through SQLAlchemy, local file storage for resumes and two optional external services. The browser never talks to the database, and the provider key is held only by the server.

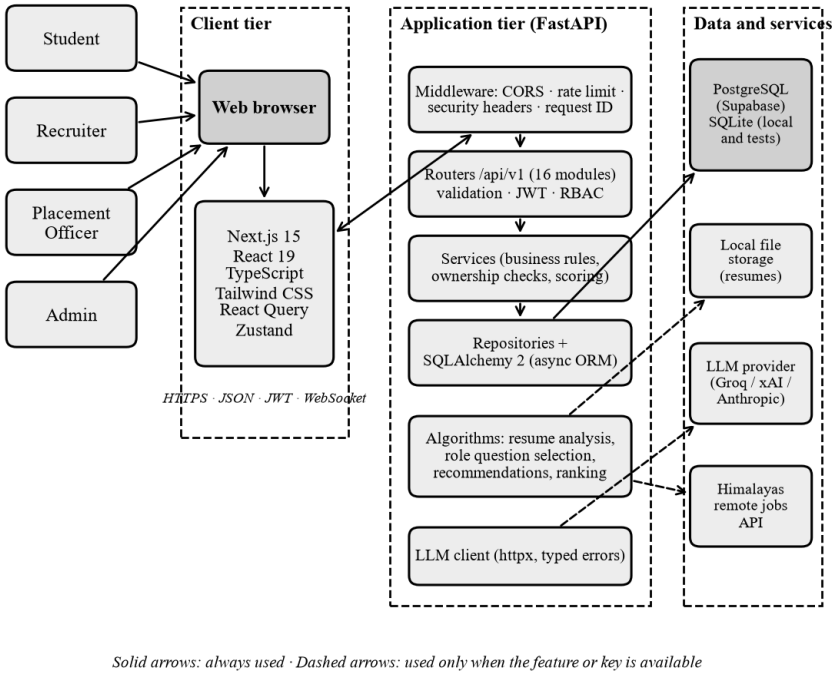


Fig. 4.1: Overall System Architecture of Career OS

Inside the backend (Fig. 4.2) every request passes through middleware (request identifier, security headers, rate limiter and cross-origin handling), a router that validates the body with Pydantic and resolves authentication dependencies, a service that applies business rules and ownership checks, and a repository that runs SQLAlchemy queries through an asynchronous session [14–16]. JSON is exchanged over HTTPS using REST conventions [17]; live interview sessions use WebSocket connections [18].

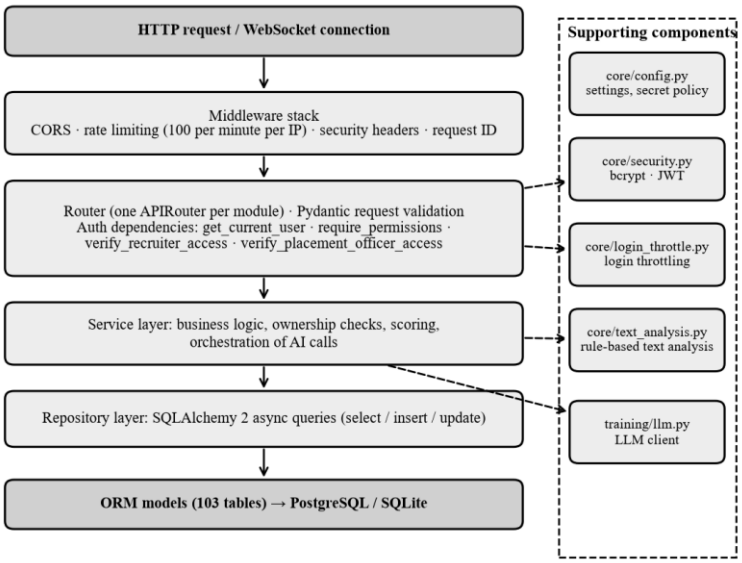


Fig. 4.2: Layered Architecture of the Backend

4.3 AUTHENTICATION AND AI PROCESSING FLOW

During login (Fig. 4.3) the server checks the login throttle (five failures per account and thirty per address in fifteen minutes), verifies the password hash with bcrypt [19] and issues a short-lived access token and a refresh token, both JSON Web Tokens [20]. Authorization then depends on the route: personal data is protected by an ownership check, 34 administrator routes need named permissions, and recruiter and officer routes need membership of a company or a college.

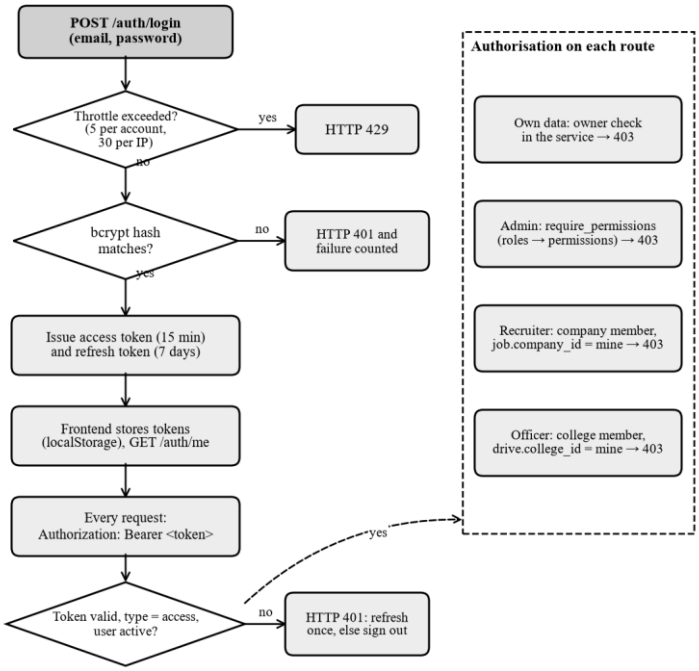


Fig. 4.3: Authentication and Authorization Flow

For an AI Mentor message (Fig. 4.4) the server verifies the user and the conversation, applies a limit of twelve messages per minute and refuses to continue when no provider key is configured, so that it never produces a pretend answer. A context aggregator reads the student's own saved records, the last ten messages are added as history and one request is sent to the provider. Only a successful reply is saved.

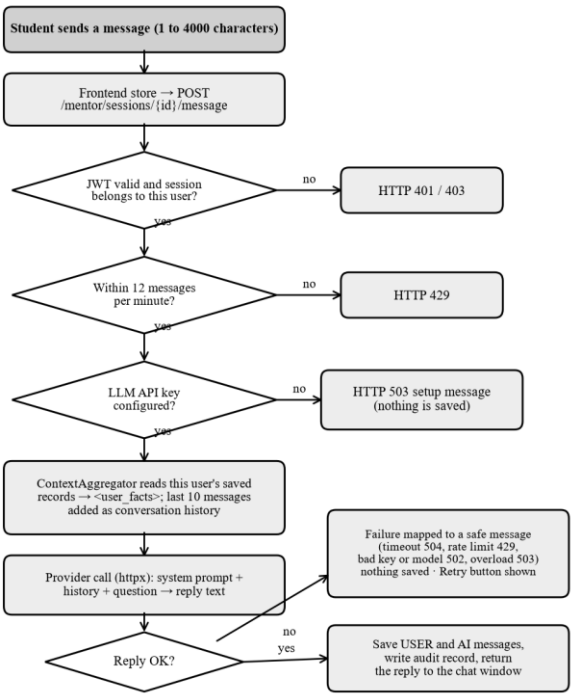


Fig. 4.4: Processing Flow of an AI Mentor Message

4.4 MODULE DESCRIPTION

Table 4.1: Summary of the Career OS Modules

Module	Purpose and processing	Limitation
User authentication	Accounts and access control; throttle, bcrypt, token issue and refresh	No e-mail verification or password reset
Dashboard and profile	Readiness is the mean of the components that have data	Transparent average, not a prediction
Resume analysis	Validate, store, extract text; score completeness, readability and wording	Rule-based; scanned PDFs flagged, not read
Job recommendations	Query the Himalayas interface; score skill overlap	Needs internet; remote roles only
Interview preparation	Round and topic practice, exact MCQ grading, rubric or optional LLM scoring	Coding answers self-assessed
Role-based training	Normalise a job title, map to topics, select unseen questions, validate generated ones	Reviewed questions for 22 roles
AI Mentor	Answer free-form questions from verified facts and bounded history	Needs a key; answers can be wrong
Communication and interview studio	Filler and hedging counts; rubric scoring over WebSocket	Text only; no speech or video
Recruiter, placement and admin	Company jobs, college drives, weighted ranking, users, roles, audit logs	Any user can create a company or college

4.5 DATA FLOW, USE CASE AND ACTIVITY DIAGRAMS

The level-1 data flow diagram (Fig. 4.5) divides the system into seven processes and seven data stores; each process reads and writes only its own tables, and the mentor reads verified facts through the context aggregator. Fig. 4.6 lists the main use cases of each actor, and the activity diagram (Fig. 4.7) follows one role-based training session from the choice of a job role to the final review.

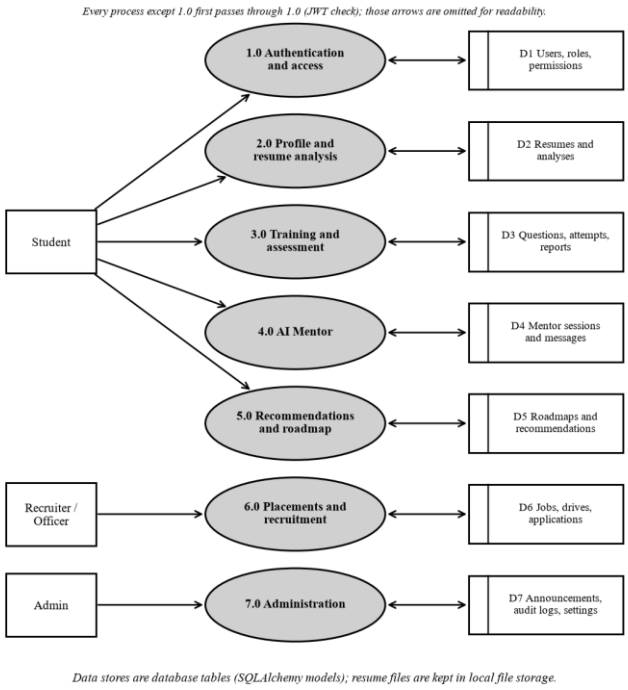


Fig. 4.5: Data Flow Diagram, Level 1

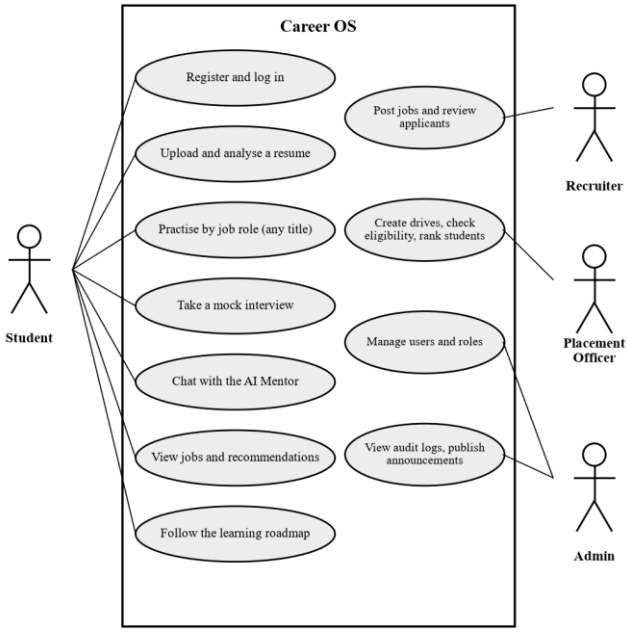


Fig. 4.6: Use Case Diagram of Career OS

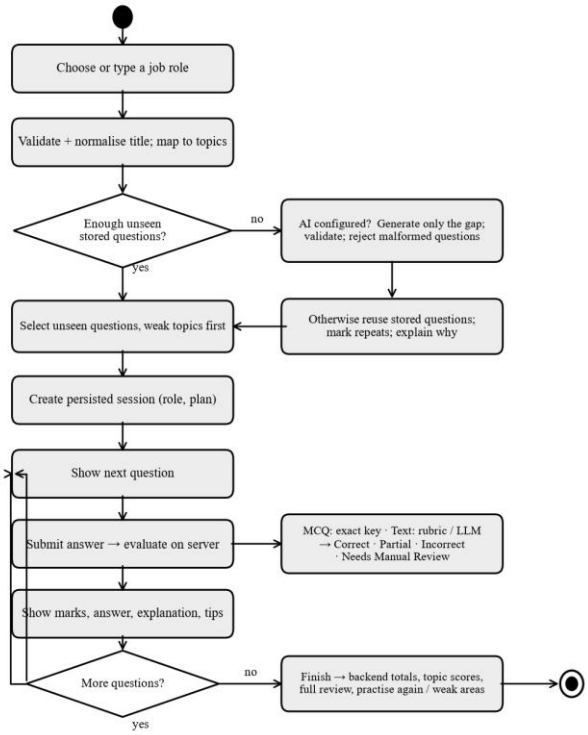


Fig. 4.7: Activity Diagram of a Role-Based Training Session

4.6 ENTITY RELATIONSHIP DIAGRAM

The database holds 103 tables, 78 foreign keys and 54 named indexes. Fig. 4.8 shows the entities central to the student journey. Every table inherits a UUID primary key and the columns created_at, updated_at and deleted_at from a common base class, and unique constraints protect one-to-one links such as profiles.user_id.

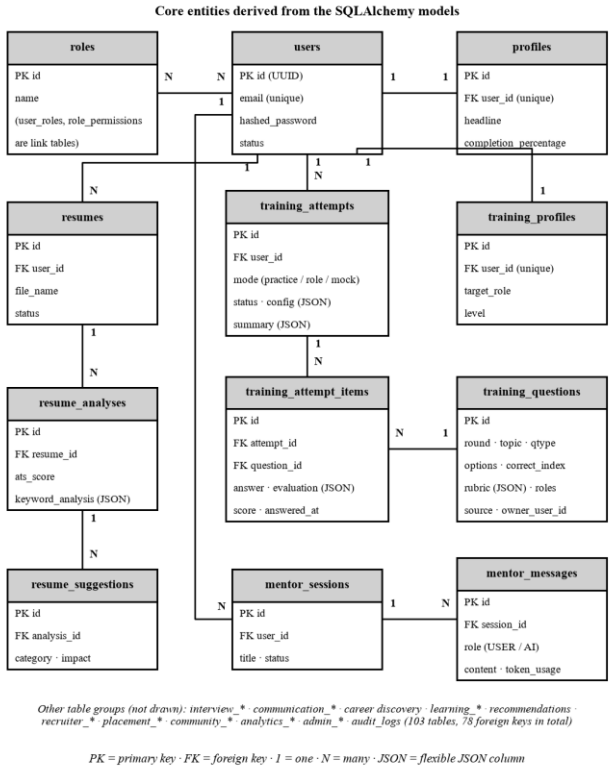


Fig. 4.8: Entity Relationship Diagram of the Core Tables

4.7 ALGORITHMS AND PSEUDOCODE

No custom model was trained and no fine-tuning is performed. Table 4.2 states exactly which features use a language model and which are ordinary algorithms.

Table 4.2: Classification of Intelligent Features

Feature	Type	LLM
AI Mentor chat	LLM-powered (needs key)	Yes
Resume extraction, scoring, skill extraction	Rule-based formulas	No
Job matching	Set-overlap matching	No
Recommendations and roadmap	Rule-based	No
Multiple-choice grading	Exact comparison	No
Written-answer grading	Rubric; LLM optional	Optional
Role question generation	LLM with validation (needs key)	Optional
Role question selection	Weighted selection algorithm	No
Placement ranking, recruiter matching	Weighted sums, overlap	No

Resume analysis. PDF text is read with pypdf (first fifteen pages) and DOCX text from the file's XML. A file with fewer than twenty readable words is flagged as unreadable, because no OCR is performed. Skills are matched against a curated dictionary with word-boundary expressions, and skills expected for the target role but not found are reported as missing. The output carries the method label rule_based_v1.

Role-based question selection. The title is cleaned and kept as typed for display, while a normalised key (seniority words removed, synonyms such as developer and engineer unified) is mapped to a catalogue role, an alias, a close variant or keyword-inferred topics.

```
FUNCTION plan_role_session(student, title, difficulty, topics, count):
    key    <- normalise(title)
    topics <- map_to_topics(key) UNION topics
    IF topics IS EMPTY AND NOT llm_key_configured: REFUSE with clear message
    chosen <- weighted_pick(unseen(bank, topics), count, adapted(difficulty))
    IF LENGTH(chosen) < count AND llm_key_configured:
        chosen <- chosen + validated(generate(title, count - LENGTH(chosen)))
    IF LENGTH(chosen) < count:
        chosen <- chosen + marked_repeats(count - LENGTH(chosen))
    RETURN session(chosen, plan_notes)
```

Validation of generated questions. A generated question is accepted only if it has a known kind, type and difficulty, does not claim to come from a real company interview, is not a near duplicate (token-overlap similarity of 0.8 or more is rejected), has four distinct options and a consistent answer index when it is multiple-choice, and at least two expected-concept groups when it is written. Accepted questions are stored for that student only and labelled unreviewed.

Table 4.3: Marking Rules of Role-Based Training

Item	Rule
Marks per question	Multiple-choice 1 mark; written answer 2 marks (quarter-mark steps)
Written-answer status	Correct at 75 percent or more, Partially Correct from 34 percent, otherwise Incorrect
Rule-based credit	Share of expected concepts found, multiplied by a length factor between 0.7 and 1
LLM credit	Mean of correctness, relevance and completeness (0 to 5), after schema validation
Unanswered	0 marks and included in the total
Final percentage	Marks earned over marks available, computed on the server from saved evaluations

CHAPTER 5

IMPLEMENTATION

5.1 IMPLEMENTATION OVERVIEW

The system was implemented with a Next.js frontend and a FastAPI backend in one Git repository: about 18,400 lines of Python in 183 files and about 13,300 lines of TypeScript in 84 files, excluding tests. Table 5.1 lists the main technologies and their roles. Dockerfiles and Kubernetes manifests exist but were not built or run during this work; Alembic, Celery, Redis and S3 storage are declared but dormant.

Table 5.1: Technologies and Their Roles

Technology	Role in Career OS
Next.js 15.5, React 19.1, TypeScript 5	All pages in app/(auth) and app/(dashboard); type check passes with zero errors
Tailwind CSS 4, React Query 5, Zustand 5	Styling with dark mode; server-state caching; session, mentor chat and notification stores
FastAPI, Uvicorn, Pydantic 2	app/main.py and 16 routers; validation of every request body
SQLAlchemy 2 (async), asyncpg, aiosqlite	Repository layer for PostgreSQL in production and SQLite locally
python-jose, bcrypt, httpx, pypdf	Tokens and password hashes; provider and jobs calls; offline resume parsing
pytest, Vitest, ESLint, ruff	483 backend tests and 54 frontend tests; code style

5.2 DATABASE IMPLEMENTATION

All persistent records are stored in a relational database of 103 tables, defined as SQLAlchemy classes in seven groups: identity and access; profile and resume; training; interviews and communication; career, learning and mentor; recruiters and placements; and community, analytics and administration. PostgreSQL [21], provided by Supabase [22], is the intended production database; SQLite is used for local runs and tests so that tests never touch a shared database. Queries are built with SQLAlchemy, so SQL strings are never assembled by hand. Tables are created at start-up only for SQLite, and scripts that write data refuse a non-local database unless a confirmation variable is set. Persistence on the remote Supabase database was deliberately not contacted and is therefore **not verified**.

5.3 BACKEND API IMPLEMENTATION

The application installs three exception handlers, four middleware components and sixteen routers under /api/v1, giving 124 routes in total (admin 35, training 21, placements 12, recruiters 8, community 8, mentor 7 and others). Invalid input is rejected with HTTP 422

before any service code runs, and provider failures are mapped to fixed, user-safe messages (504 timeout, 429 rate limit, 502 rejected credentials, 503 unconfigured provider). Table 5.2 lists the endpoints that carry the main features.

Table 5.2: Key API Endpoints

Method	Endpoint	Description
POST	/auth/register, /auth/login	Create a user; return access and refresh tokens (throttled)
POST	/resumes/upload	Validate and store the file; analysis runs in the background (202)
GET	/resumes/{id}/analysis	Scores, skills and suggestions; owner only
GET	/jobs/recommended	Fetch remote jobs and score them against saved skills
POST	/mentor/sessions/{id}/message	Call the LLM; save a successful exchange
POST	/training/role-sessions	Fresh questions plus a plan for a role
POST	/training/role-sessions/{id}/answer	Instant evaluation; no double scoring
GET	/training/role-sessions/{id}/review	Question-by-question review with filters
WS	/interviews/ws/{session_id}	Live interview events START, ANSWER_SUBMITTED, FINISH

5.4 RESUME ANALYSIS PIPELINE

The upload endpoint returns immediately with status 202 and the analysis runs in a background task (Fig. 5.1), so the interface stays responsive. The analysis endpoint returns data only if the resume belongs to the signed-in user.

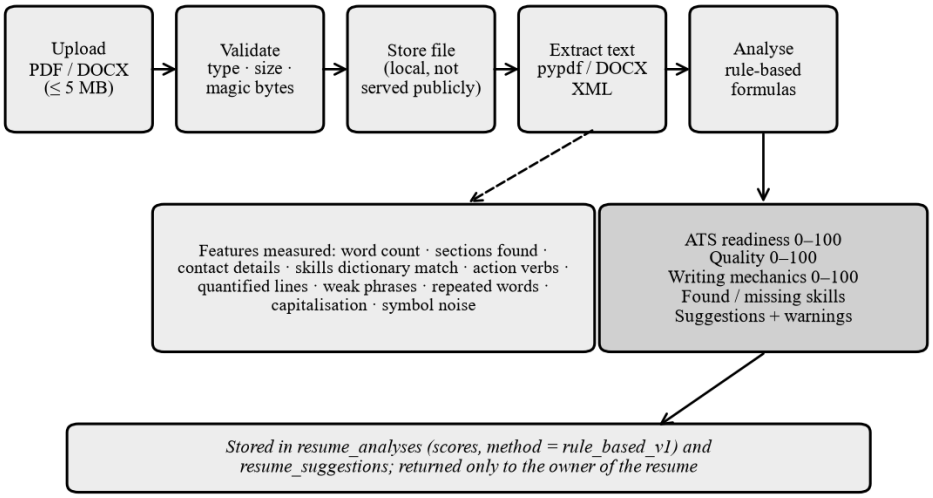


Fig. 5.1: Resume Analysis Pipeline

5.5 USER INTERFACE AND SCREENS

The screenshots below were captured from the running application. The data belongs to fictional users created through the application's own interface; no real personal data and no remote database were used. The login page shows server messages such as a throttling notice,

and the dashboard shows career readiness, the latest ATS resume score, learning progress and the latest interview score.



Fig. 5.2: Login Page (Left) and Registration Page (Right)

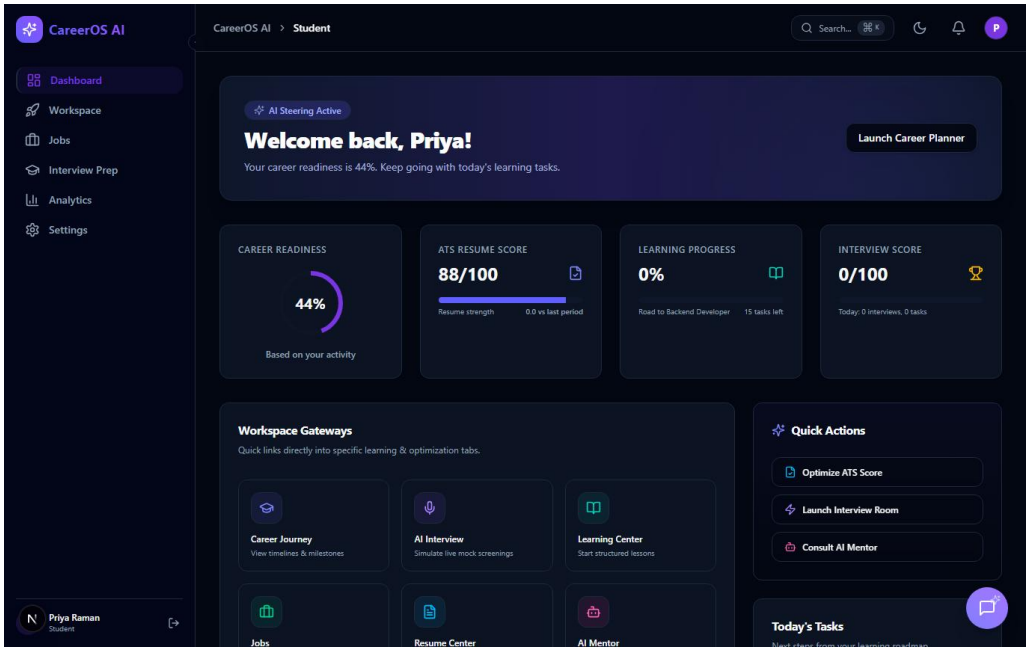


Fig. 5.3: Student Dashboard

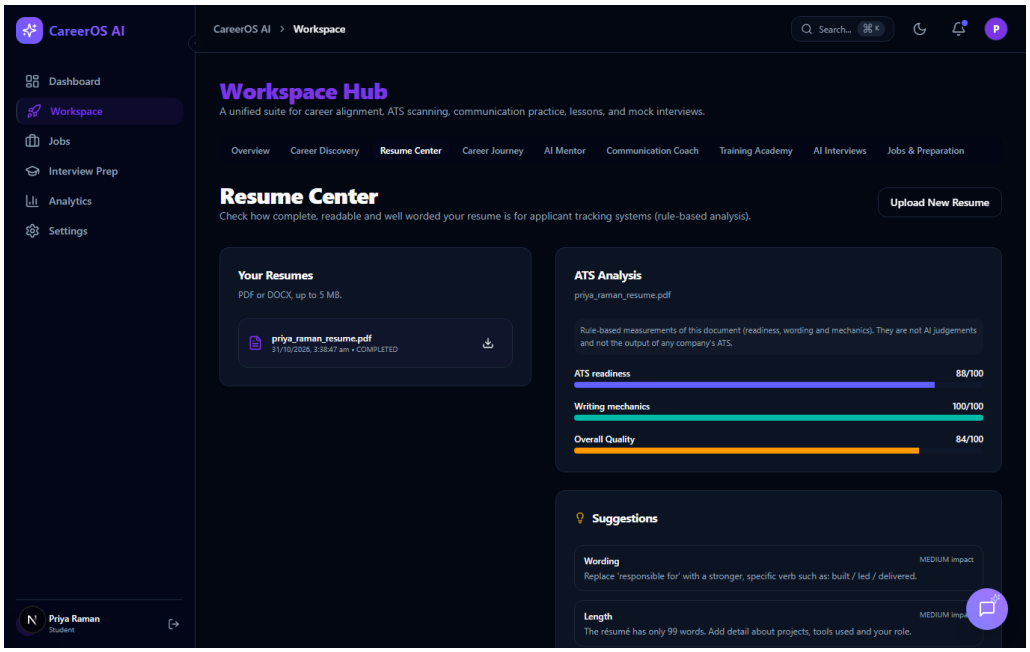


Fig. 5.4: Resume Center with the Rule-Based Analysis

The interview-preparation page is organised in tabs for the roadmap, role training, aptitude, technical, coding, communication, HR, mock interviews and performance, and shows the overall preparation percentage computed from saved answers. In role training the student chooses or types a job title; after each answer the status, marks, correct answer and explanation appear immediately, and the final review shows topic-wise bars, strong and weak areas and the Practise Again and Improve Weak Areas buttons.

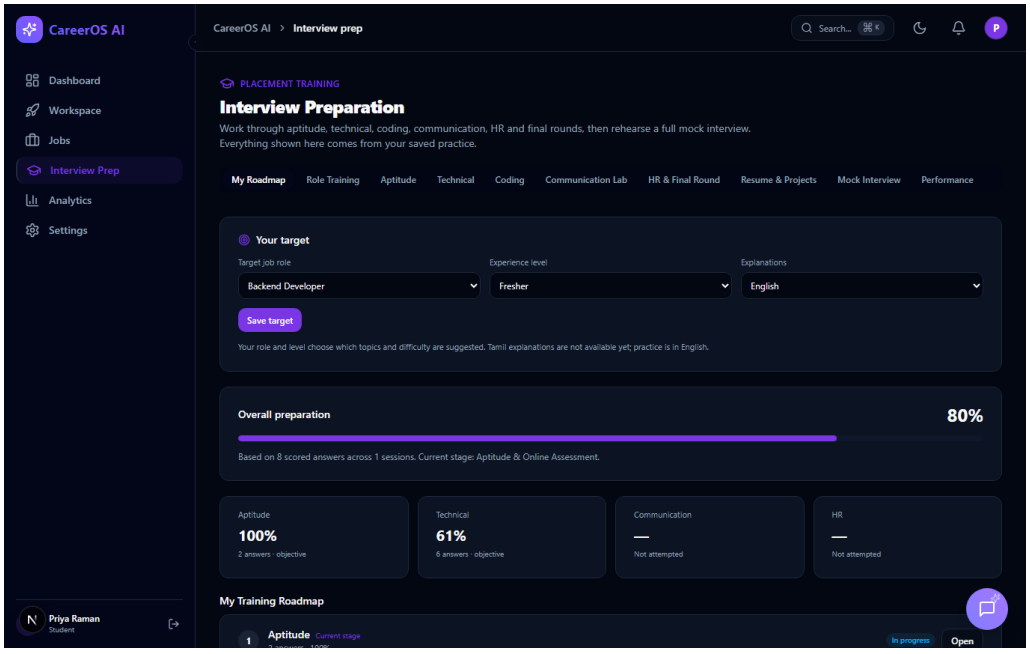


Fig. 5.5: Interview Preparation Home with Progress by Round

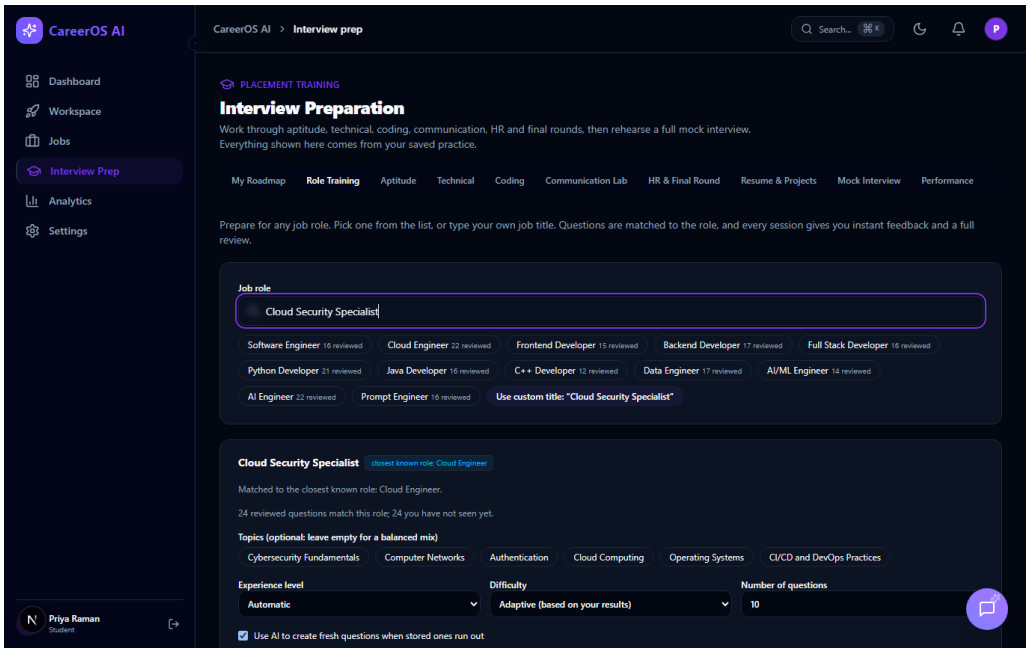


Fig. 5.6: Role Selection with a Custom Job Title

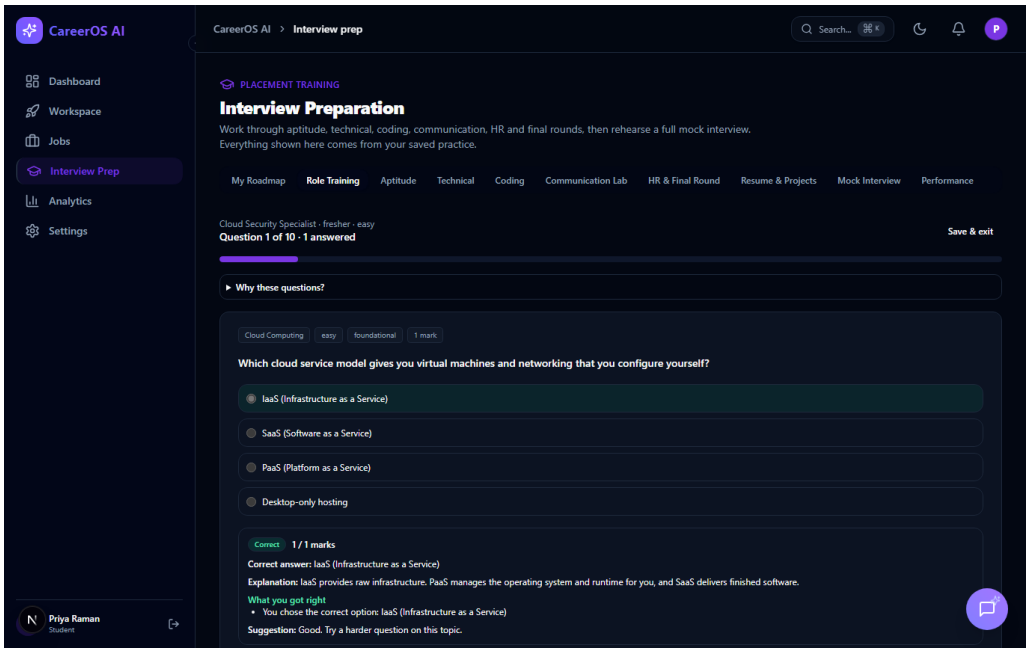


Fig. 5.7: Immediate Feedback After Submitting an Answer

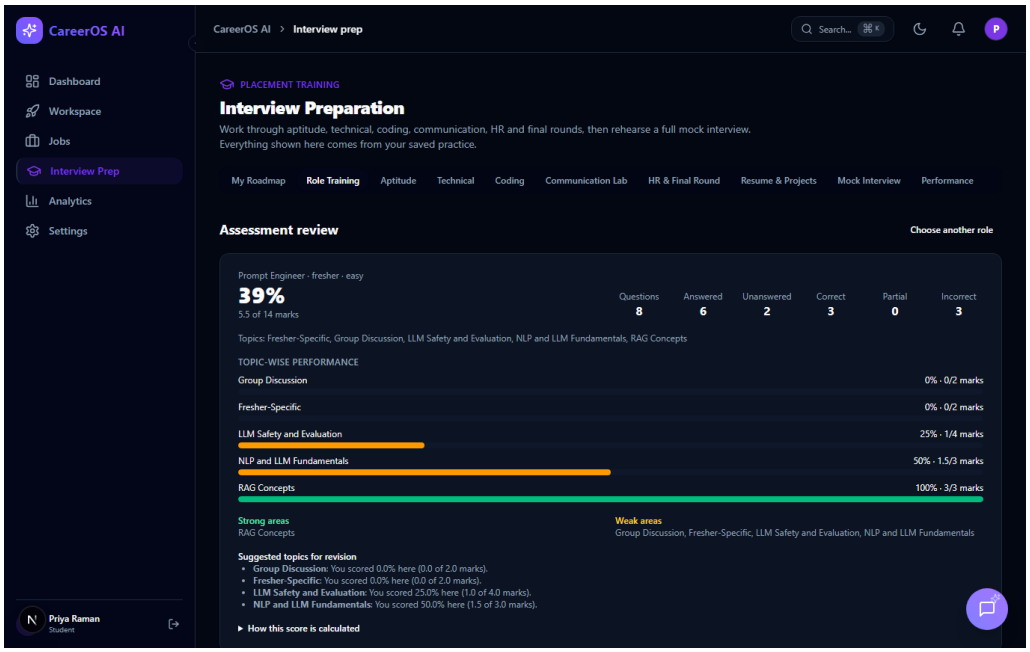


Fig. 5.8: Assessment Review: Score, Counts and Topic-Wise Performance

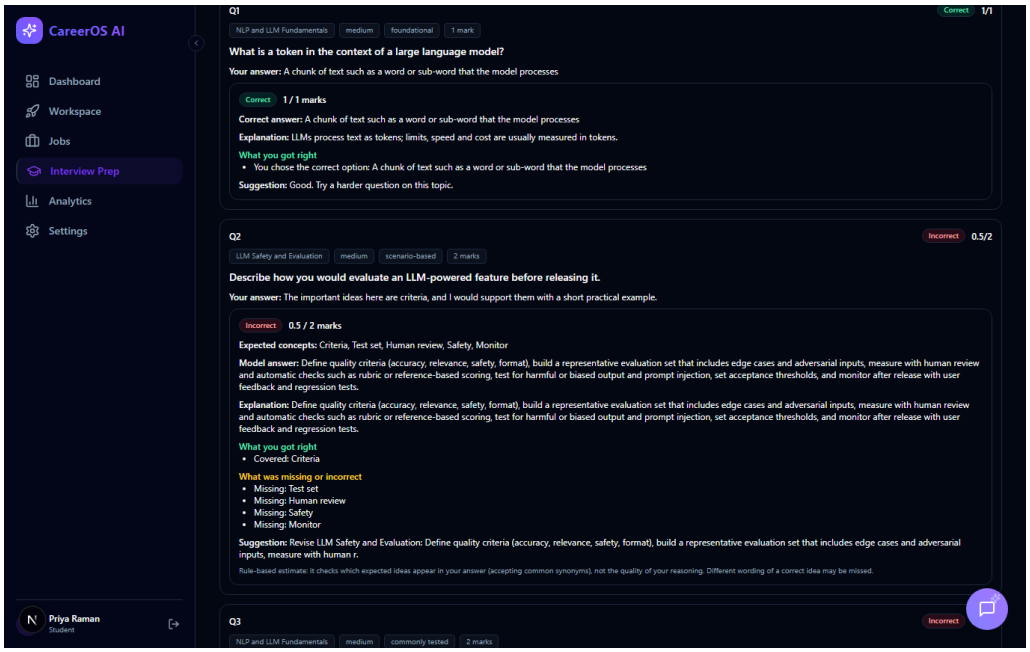


Fig. 5.9: Question-by-Question Review with Model Answer and Missing Concepts

The mentor page shows the conversation and a separate panel of suggested next steps; without a provider key it shows a setup notice instead of pretending to answer. The recommended-jobs tab lists live remote listings with a skill-match estimate. The Communication Coach gives text-based feedback on tone, clarity, filler words and writing mechanics for a chosen scenario; Career Discovery asks a short questionnaire about interests; and the Training Academy generates a roadmap of modules and tasks for the target role. Recruiters, placement officers and administrators have their own portals.

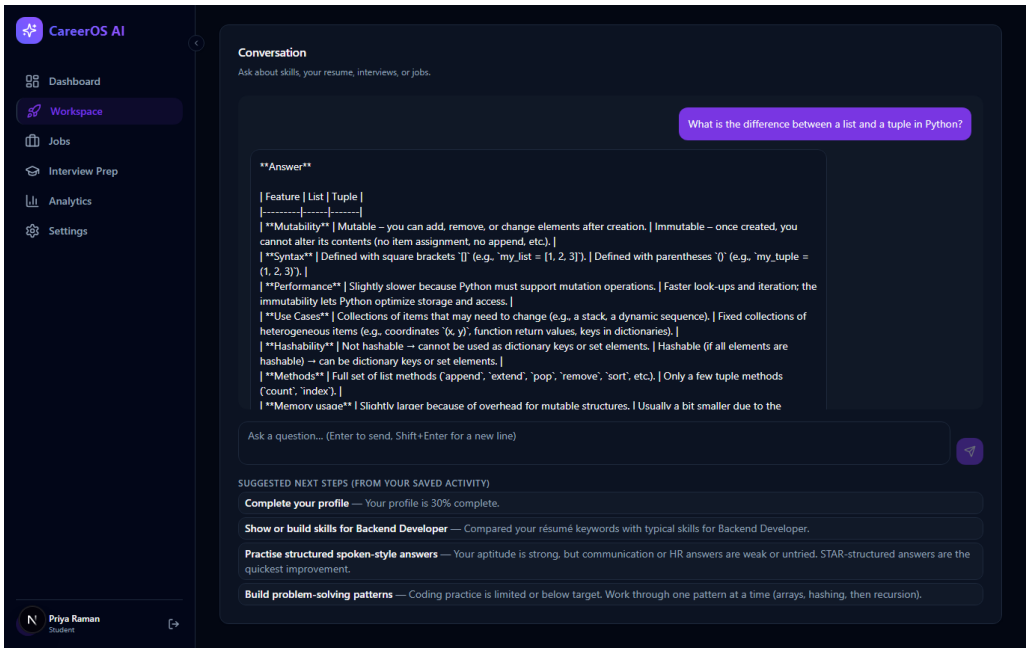


Fig. 5.10: AI Mentor Conversation with Suggested Next Steps

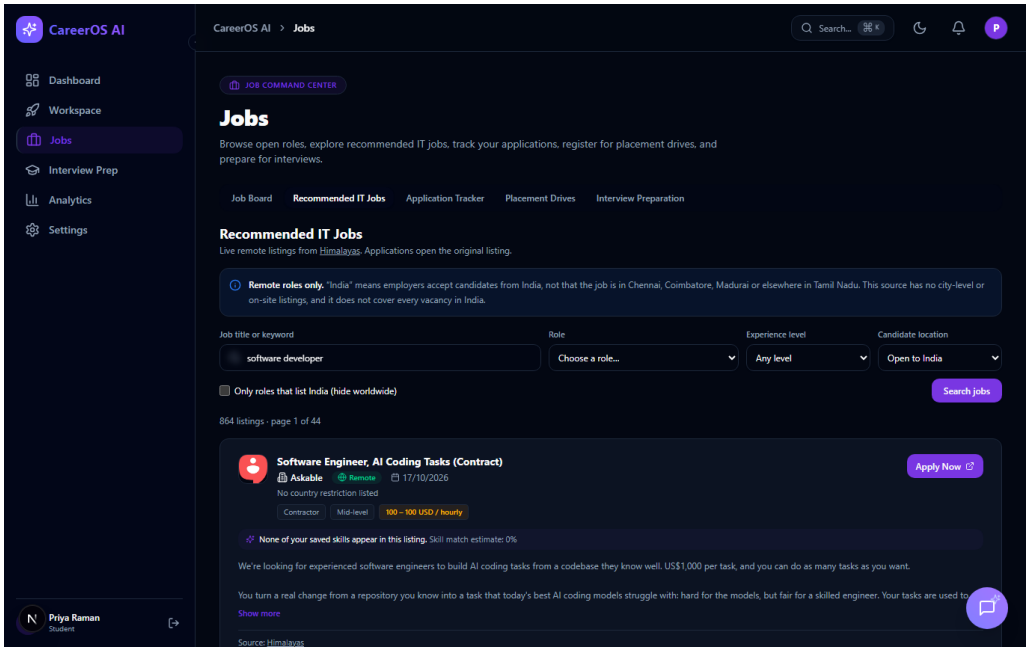


Fig. 5.11: Recommended IT Jobs from a Live Remote-Jobs Source

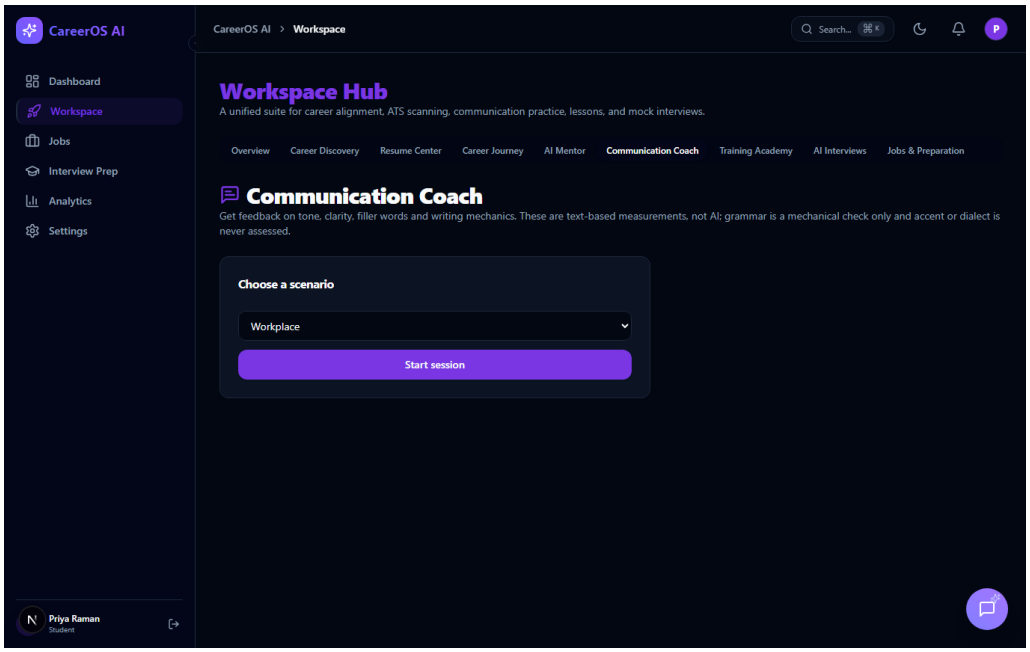


Fig. 5.12: Communication Coach Scenario Selection

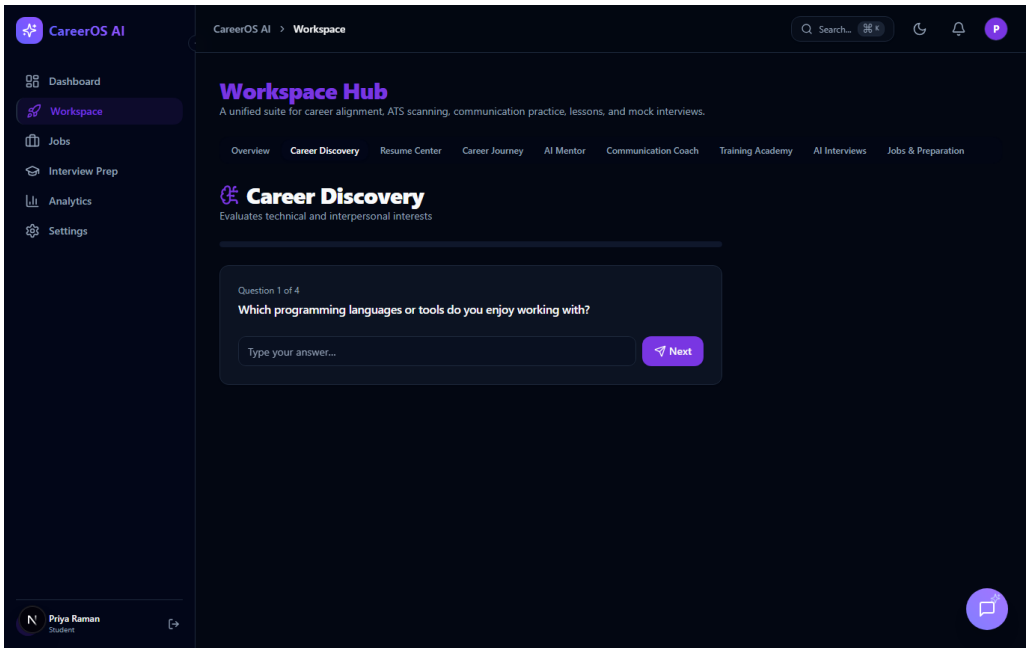


Fig. 5.13: Career Discovery Questionnaire

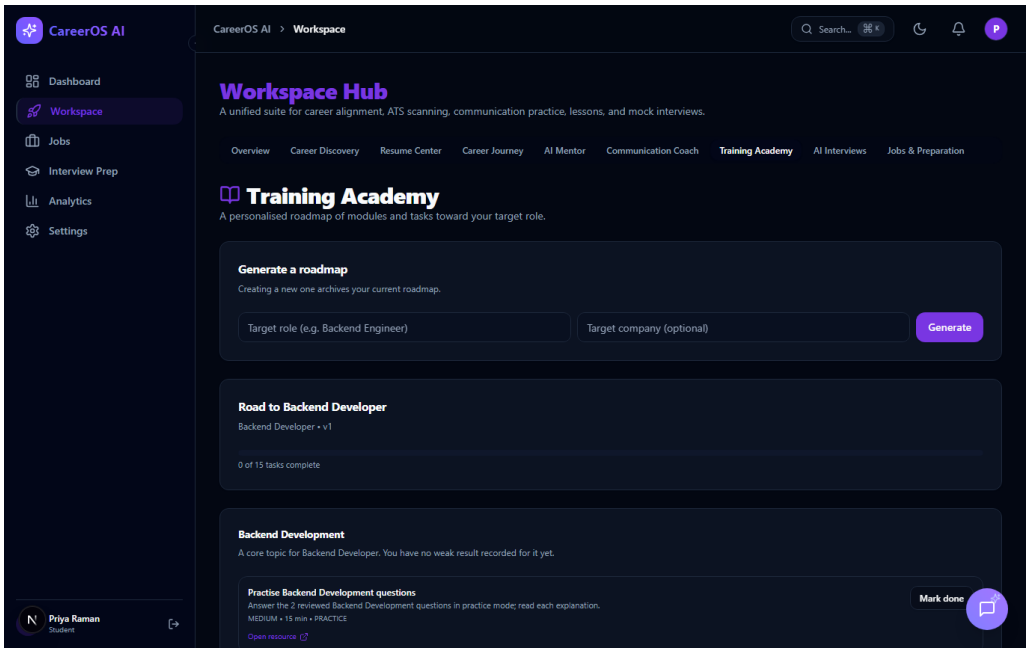


Fig. 5.14: Learning Roadmap in the Training Academy

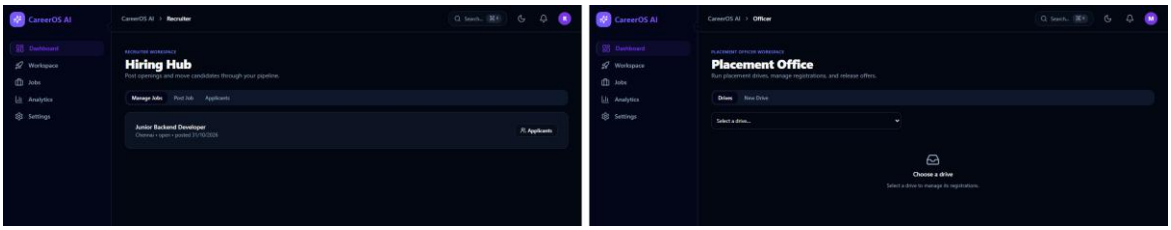


Fig. 5.15: Recruiter Portal (Left) and Placement Officer Portal (Right)

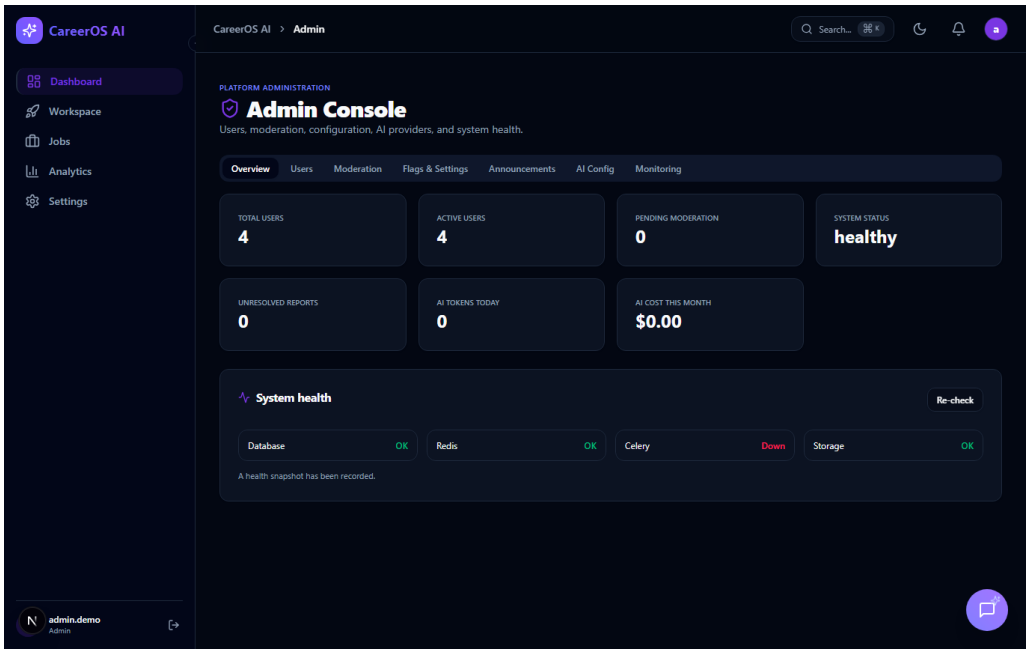


Fig. 5.16: Administrator Console

5.6 TESTING STRATEGY

Tests use an in-memory SQLite database and scripted language-model responses, never the remote database and never a real provider. API tests call the real application through

an HTTP test client; security tests verify that unauthenticated requests receive 401, that logins are throttled and that another student's resume, conversation or session cannot be read. Table 5.3 lists selected test cases.

Table 5.3: Test Cases and Expected Outcomes

ID	Test case	Expected result	Status
TC1	Register with a password shorter than 8 characters	Rejected with 422	PASSED
TC2	Log in with six wrong passwords	Sixth attempt blocked (429)	PASSED
TC3	Student B downloads Student A's resume	403, no file returned	PASSED
TC4	Mentor request when the provider fails	Safe error; no message saved	PASSED
TC5	Role training for "Cloud Security Specialist"	Title kept; matched to Cloud Engineer	PASSED
TC6	Unmapped title without an LLM key	Refused with a clear message	PASSED
TC7	Second session for the same role	Prefers unseen questions	PASSED
TC8	Written answer with half of the concepts	Partially Correct with partial marks	PASSED
TC9	Submit the same answer twice	Saved result returned; not scored twice	PASSED
TC10	Malformed generated questions (11 kinds)	All rejected before storage	PASSED

CHAPTER 6

RESULTS AND DISCUSSION

6.1 STATUS OF RESULTS

The project delivered a working full-stack platform with 124 backend routes, 103 database tables, a reviewed bank of 235 questions, sixteen backend modules and role-specific portals. Registration, login, access control, resume analysis, practice with immediate grading, role-based training for any job title, the AI Mentor (with a provider key) and the recruiter, placement officer and administrator portals work end to end. Recommended jobs depend on a public third-party interface, and the analytics and community screens are functional but basic.

6.2 TEST EXECUTION RESULTS

The automated suites were executed on 31 October 2026. The backend suite contains 483 tests in six layers (Fig. 6.1), and the frontend suite contains 54 tests in eight files. Table 6.1 summarises the verification results.

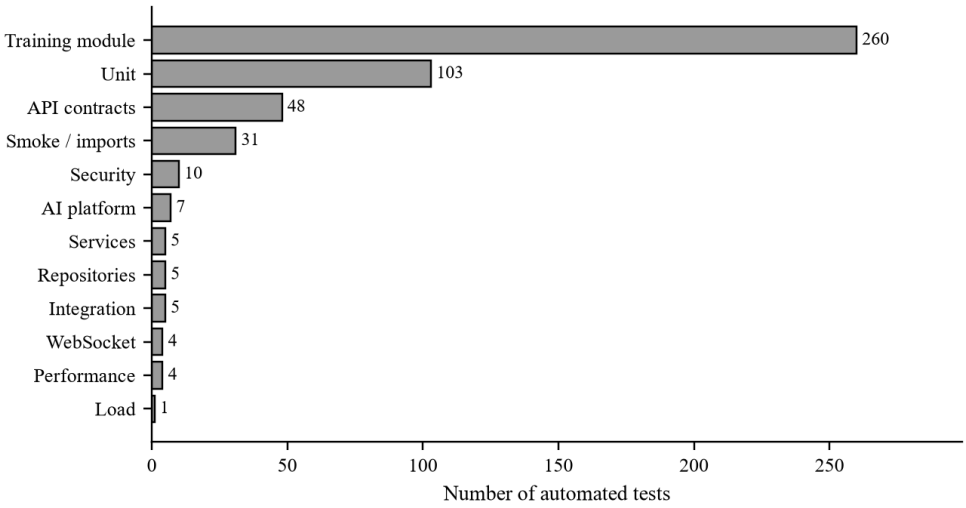


Fig. 6.1: Number of Backend Tests in Each Layer

Table 6.1: Summary of Verification Results (Final Run, 31 October 2026)

Check	Command	Result	Status
Backend tests	python -m pytest	483 tests: 481 passed, 2 failed	FAILED (2)
Frontend tests	npx vitest run	8 files, 54 tests passed	PASSED
TypeScript	npx tsc --noEmit	0 errors	PASSED
Lint (frontend)	npm run lint	0 errors, 2 warnings	PASSED
Production build	npm run build	Compiled; 32 static pages	PASSED
Lint (backend)	ruff check app	6 import-order findings (E402)	FAILED (style)
Browser e2e scripts	-	Not executed	NOT RUN

Two backend tests failed. test_password_verify_performance is a timing benchmark that failed under full-suite load and passed in three separate reruns. test_resume_repository_save_analysis_persists_suggestions_after_analysis fails because of an uncommitted modification of the resume repository in the working tree; either the test or the modification must be updated before submission. The language-model features were also checked manually against the configured provider on 27 October 2026: the mentor answered three questions including a follow-up, and for the custom role “Sound Engineer” five generated questions passed validation.

6.3 DISCUSSION OF THE FINDINGS

Computing every score and recommendation from saved records, and keeping language-model output bounded and validated, produces feedback that is both useful and traceable. Rule-based analysis is reproducible: identical input gives identical scores. The language model is confined to the mentor and to optional generation and grading, so a provider failure does not stop the rest of the platform. The accuracy of generated answer keys and learning outcomes for real students are not yet measured.

6.4 ADVANTAGES AND LIMITATIONS

- **Integrated:** one account and one database serve resume, practice, mentor, jobs and roadmap.
- **Role-specific:** any job title can be practised, with a preference for unseen questions.
- **Traceable scoring:** totals are computed on the server from saved evaluations and labelled by method.
- **Trustworthy mentor:** the assistant receives only verified facts and refuses to pretend when no provider is configured.

Table 6.2: Known Limitations

Area	Limitation
AI	The mentor needs an API key and internet; answers can be wrong; generated keys are checked for consistency, not truth
Assessment	Written-answer scores are estimates; coding answers are self-assessed
Resume	Keyword and structure analysis only; no OCR; scores do not match any employer system
Database	Remote persistence not verified; no migration tool
Security	Tokens in browser storage cannot be revoked; no e-mail verification or password reset
Operations	No continuous-integration pipeline; Docker and Kubernetes files unverified

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 CONCLUSION

Career OS was designed to give students one place to prepare for jobs and to receive feedback they can trust. A student can upload a resume that is analysed by documented formulas, practise from a reviewed bank, prepare for any job title through the role engine, receive immediate marks and explanations and read a complete review built from saved records. A hosted language model powers the AI Mentor and can supply extra role-specific questions, but every model output is bounded, validated and labelled. The technical achievements are a layered backend with 124 routes, ownership and permission checks, a role-to-topic engine that works for unseen job titles, server-side scoring, a validated generation pipeline and a test suite of 483 backend and 54 frontend tests. The project also records its limits openly: no model was trained, the remote database was not verified and two tests need attention.

7.2 FUTURE ENHANCEMENTS

Table 7.1: Current Limitations and Proposed Future Enhancements

No.	Current limitation	Proposed future enhancement
1	No database migrations; remote persistence not verified	Add migrations and verify the managed PostgreSQL deployment
2	No e-mail verification or password reset	Add verification, password reset and HttpOnly-cookie sessions
3	Mentor replies shown as plain text	Render Markdown and stream replies
4	Reviewed bank covers 22 roles	Extend the reviewed bank to more roles
5	No OCR, code execution or speech analysis	Add OCR, sandboxed code execution and speech analysis
6	No CI pipeline; per-process rate limits	Add CI and shared rate limiting in a store such as Redis

The API is stateless and asynchronous, so several instances can run behind a load balancer. With migrations, stronger session security and a managed database, the verified prototype can become a service that a placement cell operates for a whole batch of students.

CHAPTER 8

REFERENCES

[1] J. Fuller, M. Raman, E. Sage-Gavin and K. Hines, "Hidden workers: Untapped talent," Harvard Business School Project on Managing the Future of Work and Accenture, Sep. 2021.

[2] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.

[3] P. Jaccard, "The distribution of the flora in the alpine zone," *New Phytologist*, vol. 11, no. 2, pp. 37–50, 1912.

[4] H. L. Roediger and J. D. Karpicke, "Test-enhanced learning: Taking memory tests improves long-term retention," *Psychol. Sci.*, vol. 17, no. 3, pp. 249–255, 2006.

[5] P. Black and D. Wiliam, "Assessment and classroom learning," *Assessment in Education: Principles, Policy & Practice*, vol. 5, no. 1, pp. 7–74, 1998.

[6] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[7] T. B. Brown *et al.*, "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[8] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022.

[9] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[10] Vercel, "Next.js documentation." [Online]. Available: https://nextjs.org/docs

[11] Meta Platforms, "React documentation." [Online]. Available: https://react.dev

[12] Tailwind Labs, "Tailwind CSS documentation." [Online]. Available: https://tailwindcss.com/docs

[13] TanStack, "TanStack Query documentation." [Online]. Available: https://tanstack.com/query

[14] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com

[15] Pydantic, "Pydantic documentation." [Online]. Available: https://docs.pydantic.dev

[16] SQLAlchemy, "SQLAlchemy documentation." [Online]. Available: https://docs.sqlalchemy.org

[17] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. California, Irvine, CA, USA, 2000.

[18] I. Fette and A. Melnikov, "The WebSocket protocol," IETF, RFC 6455, 2011.

[19] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf.*, 1999, pp. 81–92.

[20] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” IETF, RFC 7519, 2015.

[21] The PostgreSQL Global Development Group, “PostgreSQL documentation.” [Online]. Available: https://www.postgresql.org/docs/

[22] Supabase, “Supabase documentation.” [Online]. Available: https://supabase.com/docs